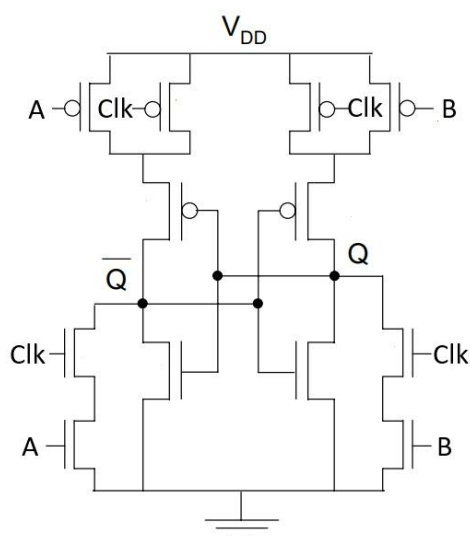


ID No:	Name:
Space for Recheck Request	

Note: The exam has three parts- Parts A,B and C. All the parts are built-in, i.e., they must be answered in the question paper ONLY. Answers written anywhere else will not be evaluated.

- Part A is closed book. Suggested time is 60 min.
- Parts B and C are open-book and can be received simultaneously on submitting Part-A.
- **casex** treats both z and x as don't care

Q1. Draw an appropriate gate-level schematic(circuit) following CMOS logic circuit. Also complete the truth table for this circuit and identify the circuit. [8M]



Gate-level Circuit diagram

Identify the circuit

Clk	A	B	Q

Q2. Implement the following function using only 8X1 mux. Assume true and complement forms of inputs are available. No logic gates are available. $F(A,B,C,D)=\sum m(0,1,2,5,8,9,10,11,13,14,15)$. [4 M]

Q3. Consider the following Verilog code. Assuming the original values of P,Q,R are xxxx (don't cares) at t=0, write the output of \$monitor statement by filling the table below the code. Specify the values in binary format ONLY. [7M]

```

module Compre(clk, a,b,c, P,Q,R);
    input [3:0] a, b, c;
    output reg [3:0] P, Q, R;
    input clk;
    always @(posedge clk)
    begin
        P <= a + b;
        Q = P + 2*c;
        R <= Q + 3*c;
    end
endmodule

module tb;
    reg [3:0] a, b, c;
    wire [3:0] P,Q,R;

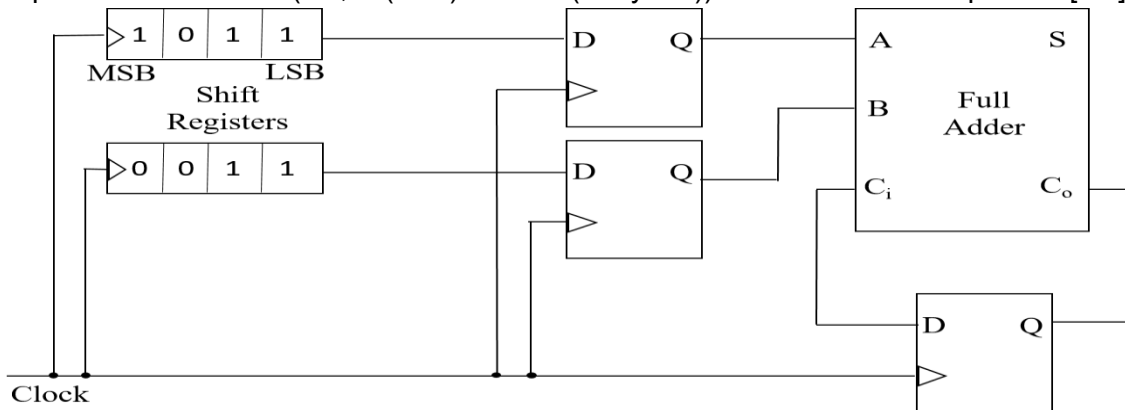
    reg clk;
    Compre cr1 (clk,a,b,c,P,Q,R);
    always @clk
    #5 clk <=~clk;

    initial begin
        $monitor($time, "a=%b,b=%b,c=%b, P=%b, Q=%b, R=%b clk=%b\n",a,b,c,P,Q,R,clk);
        a=3; b=4; c=5; clk=1'b0;
        #10 a=4; b=6;
        #10 c=7; a=3;
        #10 $finish;
    end
endmodule

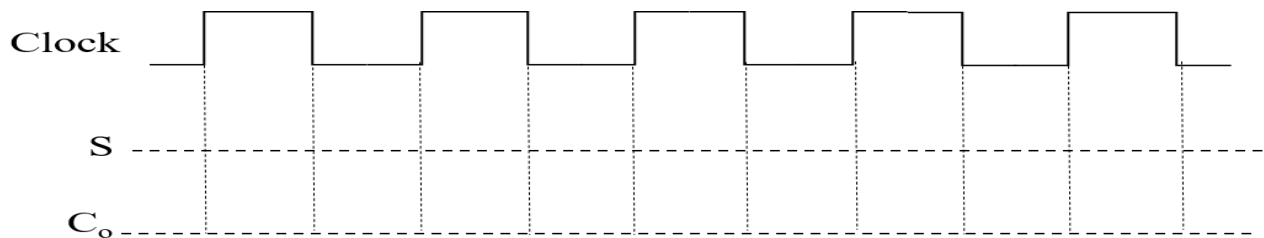
```

Time	a	b	c	P	Q	R

Q4. For the circuit shown below, two 4-bit parallel-in serial-out shift registers loaded with the data shown are used to feed the data to a full adder. Initially, all the flip-flops are in a clear state. Show the output of the full adder (i.e., S (sum) and Co (carry out)) for the first 5 clock pulses. [4M]

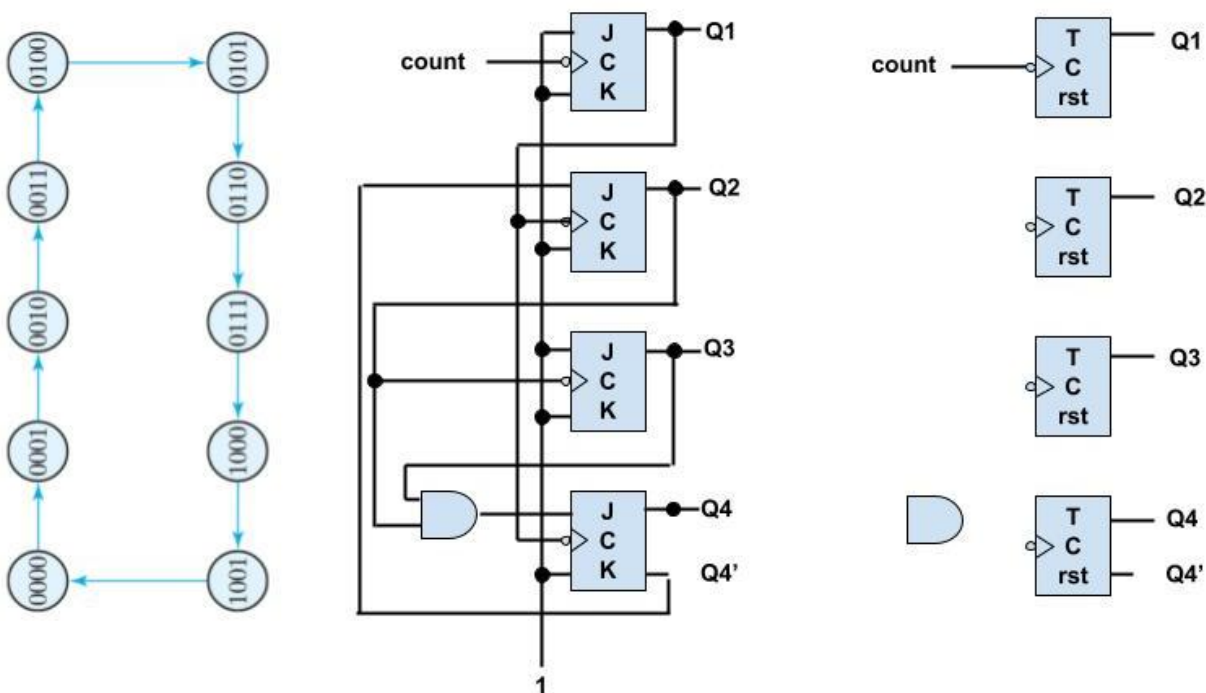


Answer:



Q5 A) Shown below is an implementation of the ripple BCD counter with JK flip-flops. Following similar logic and the same external components (a single AND gate), connect up the circuit on the right to implement a ripple BCD counter using T-flipflops with synchronous RESET.

[10M]



B) Three such counters are connected together to make a counter from 0 to 999. Each flipflop has a delay of 4 ns from the time a clock edge arrives to the time the output is complemented.

i. Which transition of the counter takes the longest time (i.e. from what number to what number)? (Specify the binary count value)

ii. What is the maximum frequency at which inputs can be received by this counter and still get counted correctly?

Q6. $F(A, B, C, D) = \sum m(1, 3, 4, 5, 9, 10, 11) + \sum d(6, 8)$ Using Quine-McCluskey technique, tabulate the PI chart for the function F. Specify the presence of a minterm in a PI using ('X'). The min term must be mentioned in terms of appropriate literals. [7 M]

min term→						
PI Term↓						

Also mention the MSP of this function obtained

F= _____

ID No:

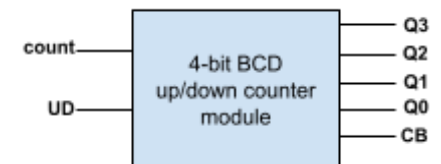
Name:

Recheck Request:

All the Questions in this part are built-in, i.e., they must be answered in the question paper ONLY. Answers written anywhere else will not be evaluated.

Q1. Parking Lot Display: We want to design a parking lot display system which tracks the incoming and exiting cars and shows the lot occupancy on a BCD display. The lot capacity is 40 cars. The system receives two sensor inputs: when a car enters, the N input becomes 1 for one clock cycle, then goes back to 0. When a car exits, the X signal becomes 1 for one clock cycle, then returns to 0. The counter cannot decrease below 0 or increase above 40. You will implement the system by designing an *UP / DOWN BCD counter* that takes a clock signal and N , X as inputs, and generates the appropriate output signals to give to the BCD display, following these steps:

A. Build a synchronous BCD up/down module: This module takes *count* and *UD* signals ($UD = 0$ for down and 1 for up) as inputs and generates a 5-bit output corresponding to the 4-bit BCD digit and a carry/borrow (CB) bit. It will have 4 D-flipflops to store the 4-bit count.



a. Complete the state table for the counter [/ 5M]:

Decimal	Q3	Q2	Q1	Q0	UD	D3	D2	D1	D0	CB
0	0	0	0	0	0					
0	0	0	0	0	1					
1	0	0	0	1	0					
1	0	0	0	1	1					
2	0	0	1	0	0					
2	0	0	1	0	1					
3	0	0	1	1	0					
3	0	0	1	1	1					
4	0	1	0	0	0					
4	0	1	0	0	1					
5	0	1	0	1	0					
5	0	1	0	1	1					
6	0	1	1	0	0					
6	0	1	1	0	1					
7	0	1	1	1	0					
7	0	1	1	1	1					
8	1	0	0	0	0					
8	1	0	0	0	1					
9	1	0	0	1	0					
9	1	0	0	1	1					
—	1									

b. Write down the expressions for [/ 2+2+4M]:

i. CB _____

ii. D0 _____

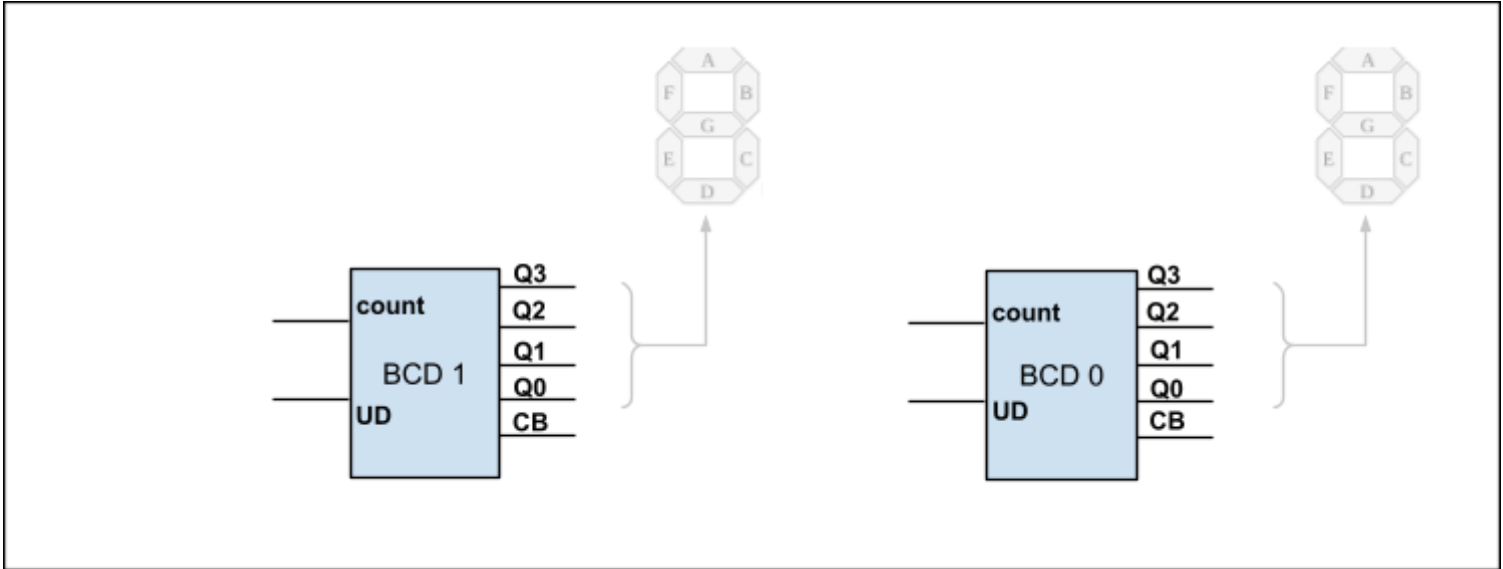
iii. D3 _____

(D3 should be in the canonical SOP form)

c. Derive the simplified equation for D1 and D2 using K-maps. *Hint: 5-variable k-maps can be simplified as two 4-variable k-maps and then combined to form a single MSP expression.*

K-maps for D1 with grouping shown clearly [/ 5M]	K-maps for D2 with grouping shown clearly [/ 5M]
MSP for D1 [/ 1M]	MSP for D2 [/ 1M]

B. Build the final circuit: Cascade two modules from above to create a 2-digit display as shown below. Determine the appropriate inputs to *count* and *UD* of both modules using the external signals *N* and *X*. For this, you will only need a single XOR, AND and NOT gate. Note that *N* and *X* can both be simultaneously 1. Use one additional AND and NOT gate each to disable counting above 40. Add your connections to the circuit shown in the box only [/ 12M].



Q2. Constant multiplier: A digital system needs to efficiently calculate the product of a 4-bit unsigned binary number, X , and the constant **6**. You have the following components at your disposal and basic AND / OR gates:

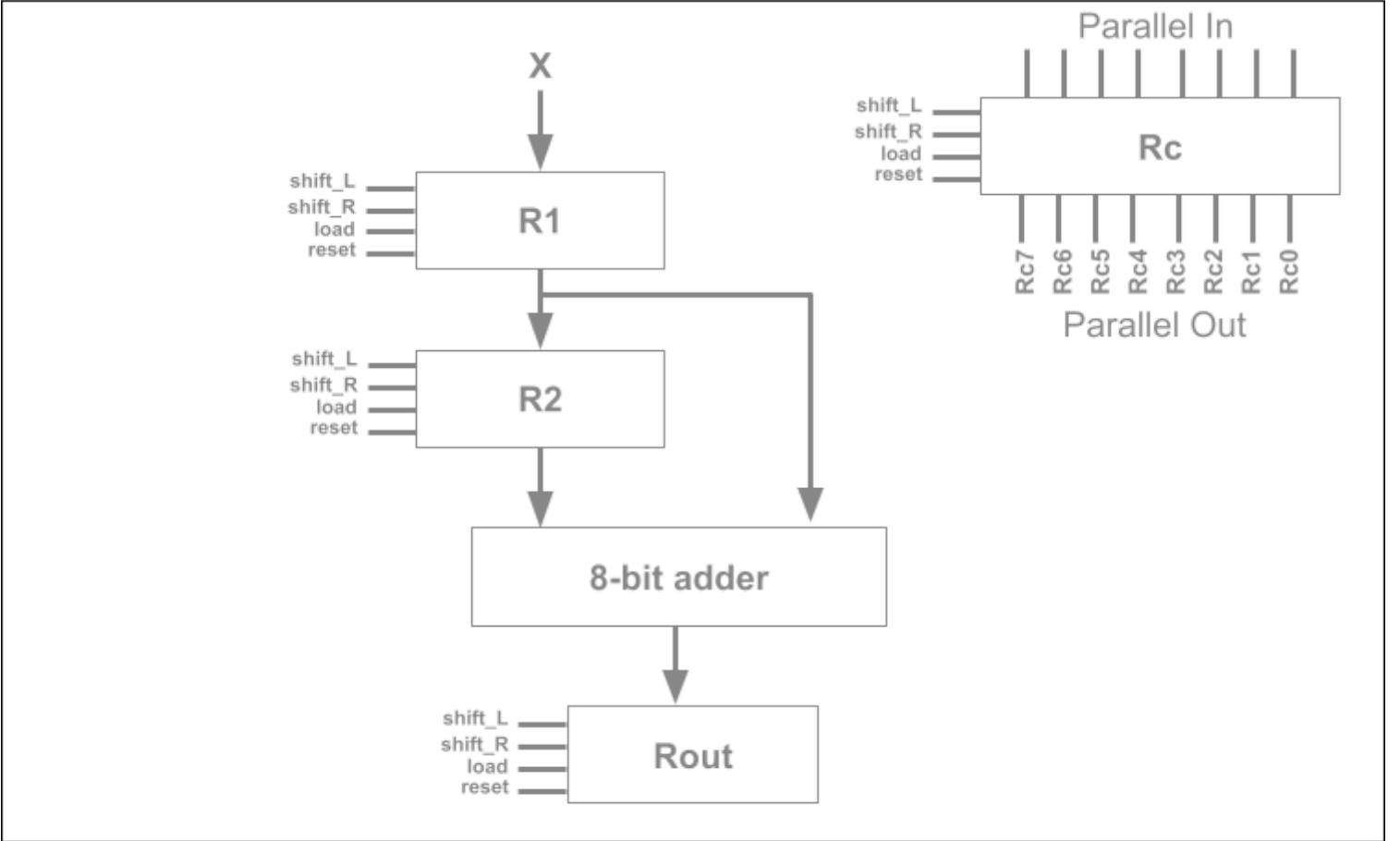
- 1. 8-bit positive edge-triggered Parallel In Parallel Out shift registers (inputs: 8-bit data, shift_L, shift_R, load, reset and clock; outputs: 8-bit data) (you may use multiple such registers named **Ri**)
- 2. 8-bit full adder

Design a circuit that will (1) load a 4-bit number into **R1**, (2) multiply it by 6, (3) store the result in **Rout**, (4) repeat.
Hint: the datapath is already provided in part B, which will help you understand the logic.

A. Control table: Complete the chart below that depicts the steps required to perform the operation. Note that each clock cycle may have more than one register operation [/ 16M].

Clock cycle	Operation	Register manipulation	Control signals required	Rc value just before the clock cycle
1	Parallel load multiplicand into R1 Shift Rc left	$R1 \leftarrow X$ $Rc \ll 1$	load (R1) = 1 shift_L (Rc) = 1	_____
2	Shift Rc left	$Rc \ll 1$	shift_L (Rc) = 1	_____
3	Shift Rc left	$Rc \ll 1$	shift_L (Rc) = 1	_____
4				_____
5				_____
6				_____
7				_____

B. Circuit: Complete the circuit below for implementing this multiplier, showing *all* missing signals / connections including any AND / OR gates that you might need for implementing the correct control logic [/ 12M].



Q3. Elevator controller: The block diagram for an elevator controller with two floors is shown on the right.

Controller inputs: The inputs FB0 and FB1 are floor buttons inside the elevator. The inputs CALL0 and CALL1 are call buttons on each floor. Together they determine whether the elevator is supposed to go to floor 0 or 1, which is stored in flip-flops N0 and N1 respectively**. R0 and R1 are signals that reset these flip-flops. The inputs FS0 and FS1 are floor switches that output a '1' when the elevator is fully at floor 0 and 1, respectively.

Controller outputs: Outputs UP and DOWN control the motor, and the elevator is stopped when $UP = DOWN = 0$. Output $DO = 1$ causes the door to open, and the door mechanism closes the doors automatically after some time, setting $DC = 1$. Doors cannot be closed from inside the elevator. The elevator should move only when the door is closed. **Note that N0 and N1 can be simultaneously 1, in which case the elevator must serve the floor it is already at before going to the other floor.

Hint: Use two flip-flops inside the controller to define the controller state ($Q1$ stores the current floor, 0 or 1, and $Q2$ stores the latest requested floor, 0 or 1; state S_{xy} indicates $Q1 = x$ and $Q2 = y$). **Complete the ASM chart for the controller.** One ASM block is already given to you. [/ 15M]

